

Dropout in Code: Regression and Classification

Contents

1. Overview	2
2. Prerequisites	3
3. Dropout for regression	3
3.1 The data and the network	3
3.2 Without dropout	3
3.3 With dropout	4
4. Dropout for classification	5
4.1 The data and the network	5
4.2 The results	5
5. Choosing the dropout rate	6
6. Practical tips	7
7. Drawbacks	7
8. Summary	9
9. Sources	9
10. Key terms	9

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Regularisation ([Note 63](#)); Overfitting ([Note 91](#)); Multi-layer perceptron (MLP) ([Note 1009](#)).
- **Compare with:** Random forest ([Note 114](#)); L1 and L2 regularisation in neural networks ([Note 1026](#)).

1. Overview

Key point: A Dropout layer after each hidden layer turns the theory into one line of Keras. On two small problems it smooths the model, shrinks the gap between training and validation loss, and improves the score on new data.

The [dropout Note](#) explained dropout: switch off a random fraction p of the nodes at every training step. This Note applies it to two small problems, one regression and one classification, and then asks how to choose p .

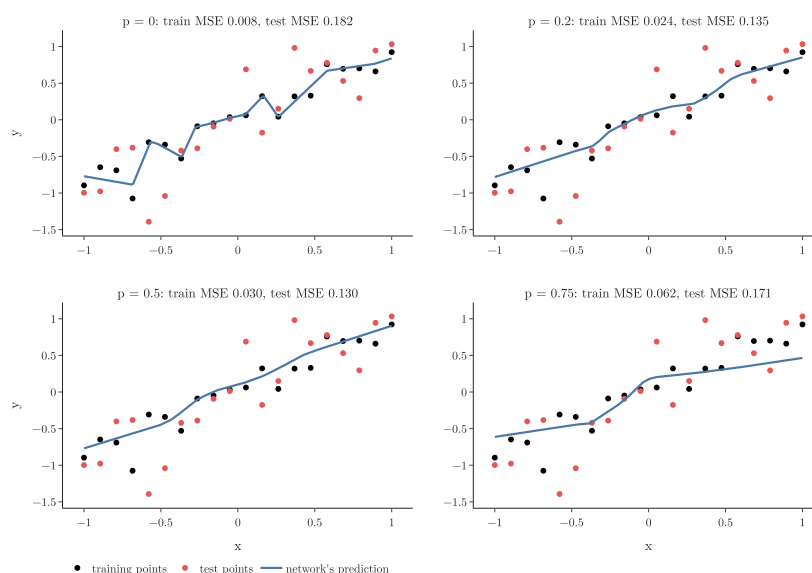


Figure 1: Regression: the network's prediction (blue) for dropout rates 0, 0.2, 0.5 and 0.75. Black: the 20 training points. Red: the 20 test points.

Figure 1 is the main result: without dropout the network bends to pass through every training point; with $p = 0.2$ or 0.5 it follows the trend; with $p = 0.75$ it no longer fits the data well. The Notebook ([notebook.ipynb](#)) runs every step.

2. Prerequisites

- The [dropout Note](#): how dropout works.
- The [customer churn Note](#), section 8: training curves.
- The [ANN for regression Note](#): a network with a linear output node and MSE loss.

3. Dropout for regression

Key point: Without dropout: training MSE 0.008, test MSE 0.182. With $p = 0.2$: 0.024 and 0.135. Training gets worse, testing gets better, and the gap shrinks.

3.1 The data and the network

Key point: 20 noisy points on the line $y = x$; two hidden layers of 128 ReLU nodes; 500 epochs.

The data is made up: 20 training points with x evenly spaced from -1 to 1 and $y = x$ plus random noise, and 20 test points at the same x values with fresh noise. The true relationship is a straight line, so a good model is close to a line.

The network is deliberately too big for 20 points: an input node, two hidden layers of 128 ReLU nodes and a linear output node. It is compiled with Adam (learning rate 0.01) and the mean squared error as loss (see the [loss functions Note](#)), and trained for 500 epochs. Training for that long on so little data invites overfitting.

3.2 Without dropout

Key point: The prediction zigzags through the training points; the test MSE is 23 times the training MSE.

	Training MSE	Test MSE
No dropout	0.008	0.182

The network fits its 20 training points almost exactly, but its error on the test points is 23 times larger. Figure 1 (top left) shows why: the blue curve bends to pass through the black training points, and these bends mean nothing for the red test points.

Figure 2 (top left) gives the second sign. The validation loss (orange) is lowest early on, then rises steadily, while the training loss (blue) keeps falling. The gap between them is overfitting.

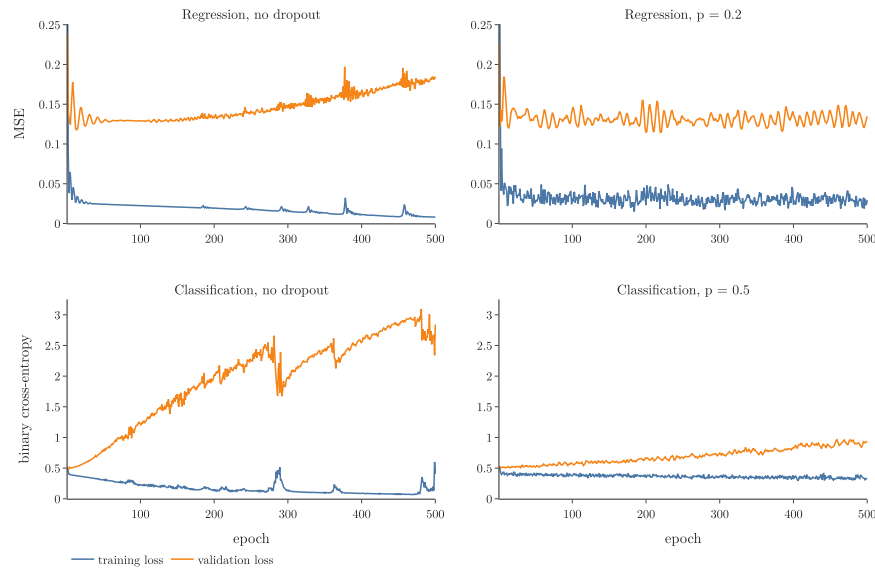


Figure 2: Training and validation loss. Top: regression without dropout and with $p = 0.2$. Bottom: classification without dropout and with $p = 0.5$.

3.3 With dropout

Key point: One Dropout(0.2) layer after each hidden layer; everything else is the same.

The new model has the same layers, with a **Dropout layer** after each hidden layer. Dropout(0.2) switches off each node of the layer before it with probability 0.2 at each training step. We do not put dropout after the output layer.

Python: The regression network with dropout.

```
model = keras.Sequential([
    keras.Input(shape=(1,)),
    keras.layers.Dense(128, activation="relu"),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(128, activation="relu"),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(1, activation="linear"),
])
model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.01),
              loss="mse")
model.fit(X_train, y_train, epochs=500,
          validation_data=(X_test, y_test))
```

A Dropout layer has no weights of its own; it only acts on the outputs of the layer before it, and only during training.

	Training MSE	Test MSE
No dropout	0.008	0.182
$p = 0.2$	0.024	0.135

The training error rose, and the test error fell by a quarter. In Figure 1 (top right) the curve is much smoother: it reacts less to the small ups and downs of the training points and follows the overall trend. In Figure 2 (top right) the validation loss no longer climbs and the gap between the curves stays the same over the 500 epochs.

Extra: The training loss that Keras prints during `fit` is computed with dropout switched on, on the shrunken sub-networks, so it is noisier and higher than the loss of the full network. The table's training MSE comes from `evaluate`, which runs the full network with dropout off (Keras FAQ, training versus testing loss). This is why the blue curve in Figure 2 (top right) is jagged.

4. Dropout for classification

Key point: Without dropout: 88% training accuracy, 71% validation accuracy and a wiggly boundary. With $p = 0.5$: 84% and 74%, a smoother boundary, and a validation loss three times smaller.

4.1 The data and the network

Key point: Two overlapping clouds of 200 points each; the same network with a sigmoid output.

The data is again made up: two clouds of points with two inputs, class 1 centred at $(0.6, 0.6)$ and class 0 at $(-0.6, -0.6)$, each with standard deviation 1. The clouds overlap, so some points of each class sit in the other's region: a straight line separates them reasonably, but not perfectly. There are 200 training points and 200 validation points.

The network is the same as in section 3, with 2 inputs and a sigmoid output node for binary classification, trained with binary cross-entropy for 500 epochs.

4.2 The results

Key point: $p = 0.2$ is not enough here; $p = 0.5$ removes most of the small islands.

Dropout rate	Training accuracy	Validation accuracy	Final validation loss
0	88%	71%	2.84
0.2	92%	71%	1.91
0.5	84%	74%	0.94

Without dropout (Figure 3, left), the boundary is irregular: it builds small islands and long fingers to capture individual points of one class in the other's region. These shapes come from the noise of the training data and will not hold on new data. The validation loss climbs from 0.5 to almost 3 (Figure 2, bottom left), while the training loss keeps falling.

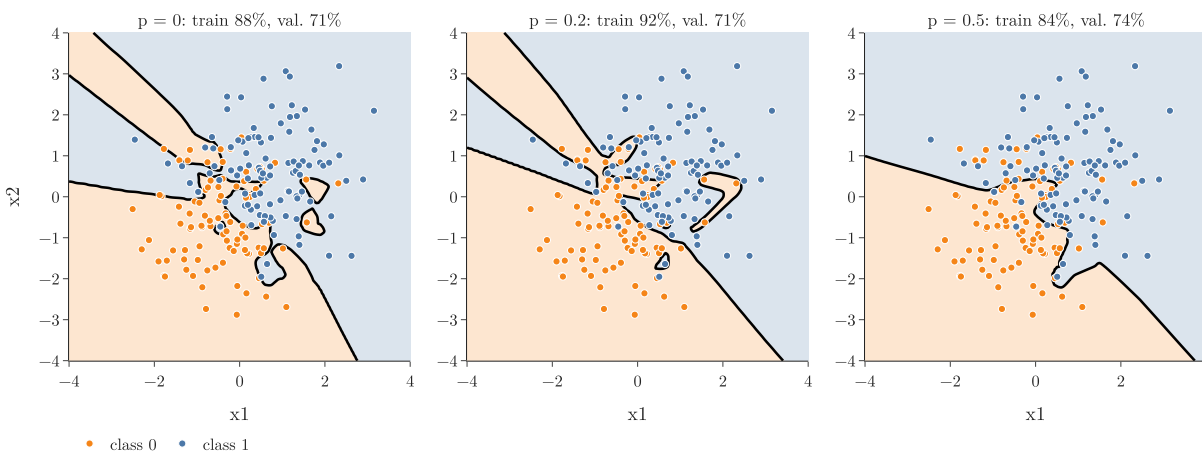


Figure 3: Classification: decision boundaries for dropout rates 0, 0.2 and 0.5, with the training points.

With $p = 0.2$ the validation loss is lower, but the boundary still has islands and the validation accuracy does not improve. The overfitting tendency is reduced, not removed.

With $p = 0.5$ the boundary (Figure 3, right) is much simpler: one main dividing line with only small bends near the overlap. The validation accuracy rises from 71% to 74%, and the validation loss falls from 2.84 to 0.94. In Figure 2 (bottom right) the gap between the curves still grows slowly, but far less than without dropout.

Dropout is a form of regularisation; because it works by random choices, it is sometimes called regularisation by randomisation.

5. Choosing the dropout rate

Key point: Too small a p leaves overfitting; too large a p underfits. Try values between 0.2 and 0.5.

Figure 1 shows all four rates on the regression problem:

Dropout rate	Training MSE	Test MSE	In Figure 1
0	0.008	0.182	zigzags through the training points: overfitting
0.2	0.024	0.135	smoother
0.5	0.030	0.130	smoother still, close to a line
0.75	0.062	0.171	too flat at the right end: underfitting

1. **In words:** as p grows, the training error always rises. The test error first falls, then rises again.
2. **The rule:** small p overfits; large p underfits. The best p is in between.
3. **Example:** here the test MSE is lowest at $p = 0.5$ (0.130). At $p = 0.75$, three out of four nodes are dropped at each step, the curve can no longer reach the points at the right end, and the test MSE climbs back to 0.171.

The same holds for classification: $p = 0.2$ was too little, $p = 0.5$ worked. The dropout rate is a hyperparameter, so the final choice comes from trying a few values.

6. Practical tips

Key point: Raise p if the network overfits, lower it if it underfits; try dropout on the last hidden layer first; typical rates are 0.2 to 0.5.

1. **Overfitting: increase p . Underfitting: decrease p .** This follows directly from section 5.
2. **Start with the last hidden layer.** Many well-performing architectures put dropout only after the last hidden layer. So rather than adding dropout everywhere at once, try it there first, check the effect, then add it to other layers.
3. **Typical rates by type of network:**

Network	Typical dropout rate
Convolutional networks (CNNs)	40% to 50%
Recurrent networks (RNNs)	20% to 30%
ANNs (multi-layer perceptrons)	10% to 50%, depending on the problem

Rates above 50% rarely help.

Extra: The original dropout paper used a lower rate on the input layer than on the hidden layers: it kept input nodes with probability 0.8 ($p = 0.2$) and hidden nodes with probability 0.5 ($p = 0.5$) (Srivastava et al. 2014, Appendix A.4). Dropping too many inputs throws away information that no later layer can recover.

7. Drawbacks

Key point: Training converges more slowly, and the loss changes from step to step, which makes gradients harder to check and the network harder to debug.

Dropout has two main costs:

1. **Slower convergence.** Each step trains only part of the network, so the network takes longer to reach good weights and biases. Experiments show training with dropout needs more epochs.

2. **A loss that keeps changing.** The loss is computed from the whole network, but at each step a different set of nodes is missing, so in effect the loss function itself changes every step. That makes the gradients hard to interpret and debugging harder: when training goes wrong, it is difficult to tell whether something is broken or the loss is just jumping around (compare the jagged blue curves of Figure 2).

Apart from these, dropout has few downsides, and it usually helps. For the mathematics, the original paper (Srivastava et al. 2014, about 30 pages) is worth reading once.

8. Summary

	No dropout	$p = 0.2$	$p = 0.5$	$p = 0.75$
Regression: training MSE	0.008	0.024	0.030	0.062
Regression: test MSE	0.182	0.135	0.130	0.171
Classification: training accuracy	88%	92%	84%	
Classification: validation accuracy	71%	71%	74%	

- In Keras, dropout is a `Dropout(p)` layer placed after the layer whose nodes it drops; never after the output layer.
- Dropout raises the training error and lowers the test error: smoother curves and boundaries, smaller gaps between the training curves.
- Small p overfits, large p underfits; try 0.2 to 0.5, starting with the last hidden layer.
- The price: slower training and a loss that is harder to monitor.

9. Sources

- Srivastava, Hinton, Krizhevsky, Sutskever and Salakhutdinov, “Dropout: A Simple Way to Prevent Neural Networks from Overfitting”, *JMLR*, 2014, Appendix A.4 (dropout rates).
- Keras FAQ, “Why is my training loss much higher than my testing loss?” (dropout is off at test time; the training loss is averaged over the epoch’s batches).

10. Key terms

Term	Meaning
Dropout layer	Keras layer that switches off a fraction p of the previous layer’s outputs at each training step; inactive at prediction
Regularisation by randomisation	A name for dropout: reducing overfitting through random choices during training